JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.function

Interface IntUnaryOperator

Functional Interface:

This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.

```
@FunctionalInterface
public interface IntUnaryOperator
```

Represents an operation on a single `int`-valued operand that produces an `int`-valued result. This is the primitive type specialization of `UnaryOperator` for `int`.

This is a functional interface whose functional method is `applyAsInt(int)`.

Since:

1.8

See Also:

`UnaryOperator`

Method Summary

Abstract Methods **Instance Methods** **Default Methods**

Modifier and Type	Method	Description
default `IntUnaryOperator`	`andThen(IntUnaryOperator after)`	Returns a composed operator that first applies

		this operator to its input, and then applies the `after` operator to the result.
`int`	**`applyAsInt`**`(int operand)`	Applies this operator to the given operand.
`default` **`IntUnaryOperator`**	**`compose`**`(`**`IntUnaryOperator`**` before)`	Returns a composed operator that first applies the `before` operator to its input, and then applies this operator to the result.
`static` **`IntUnaryOperator`**	**`identity`**`()`	Returns a unary operator that always returns its input argument.

Method Detail

applyAsInt

```
int applyAsInt(int operand)
```

Applies this operator to the given operand.

Parameters:

`operand - the operand`

Returns:

`the operator result`

compose

```
default IntUnaryOperator compose(IntUnaryOperator before)
```

Returns a composed operator that first applies the `before` operator to its input, and then applies this operator to the result. If evaluation of either operator throws an exception, it is relayed to the caller of the composed operator.

Parameters:

`before - the operator to apply before this operator is applied`

Returns:

`a composed operator that first applies the before operator and then applies this operator`

Throws:

`NullPointerException - if before is null`

See Also:

`andThen(IntUnaryOperator)`

andThen

```
default IntUnaryOperator andThen(IntUnaryOperator after)
```

Returns a composed operator that first applies this operator to its input, and then applies the `after` operator to the result. If evaluation of either operator throws an exception, it is relayed to the caller of the composed operator.

Parameters:

`after - the operator to apply after this operator is applied`

Returns:

a composed operator that first applies this operator and then applies the after operator

Throws:

`NullPointerException` - if after is null

See Also:

`compose(IntUnaryOperator)`

identity

```
static IntUnaryOperator identity()
```

Returns a unary operator that always returns its input argument.

Returns:

a unary operator that always returns its input argument

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.